Practical - 1

Aim :-
Write a Python program to demonstrate applications of Artificial Intelligence (AI) in Healthcare, Finance, and Education.

Theory :-
Artificial Intelligence (AI) is a branch of computer science that enables machines to perform tasks that normally requires human intelligence, such as learning, reasoning, problem-solving, and decision-making. AI is widely used in various domains to improve efficiency, accuracy, and automation.

Applications of AI :
- Healthcare : Disease prediction, medical image analysis, virtual assistants, drug discovery.
- Finance : Fraud detection, credit scoring, stock market prediction, customer support chatbots.
- Education : Personalized learning, automatic grading, intelligent tutoring systems, attendance management.
- Entertainment : Movie recommendation system, music recommendation, video games, content creation, video and image generation.
- E-Commerce : Product Recommendation, personalized shopping experience, chatbots for customers, fraud detection, Inventory management.

Algorithm :

1. Start the Program
2. Define separate functions for Healthcare, Finance, Education, Entertainment and E-Commerce.

3. Display AI applications in each domain.
4. Call all the functions.
5. Stop the program.

Program :

```
# Practical 1
# AI Applications in Healthcare, Finance, Education,
  Entertainment and E-Commerce

def healthcare():
    print("\n AI in Healthcare")
    print("1. Disease Prediction")
    print("2. Medical Image Analysis")
    print("3. Virtual Health Assistant")
    print("4. Drug Discovery")
    print("5. Patient Monitoring")

def finance():
    print("\n AI in Finance")
    print("1. Fraud Detection")
    print("2. Credit Score Analysis")
    print("3. Stock Market Prediction")
    print("4. Loan Approval")
    print("5. Chatbots for Customer Support")

def education():
    print("\n AI in Education")
    print("1. Personalized Learning")
    print("2. Smart Attendance System")
    print("3. Automatic Grading")
    print("4. Intelligent Tutoring System")
    print("5. Online Exam Evaluation")
```

```
def entertainment():
    print("\n AI in Entertainment")
    print("1. Movie Recommendation system")
    print("2. Music Recommendation")
    print("3. Video Games")
    print("4. Content Creation")
    print("5. Video and Image Generation")

def ecommerce():
    print("\n AI in E-Commerce")
    print("1. Product ~~Management~~ Recommendation")
    print("2. Personalized Shopping Experience")
    print("3. Chatbots for Customers")
    print("4. Fraud Detection")
    print("5. Inventory Management")

healthcare()

finance()

education()

entertainment()

ecommerce()
```

Result :

Thus the Python program to demonstrate the applications of Artificial Intelligence (AI) in Health care, Finance, Education, Entertainment and E-Commerce was implemented and executed successfully.

Practical - 2

Aim :
Write a program to import commonly used Python libraries and demonstrate their basic operations.

Theory :
Python provides many built-in and external libraries that simplify programming tasks. In Artificial Intelligence and Machine Learning, libraries like Numpy, Pandas and Matplotlib are widely used for numerical computation, data manipulation, and data visualization.

- Numpy : Used for numerical operations and array manipulation.
- Pandas : Used for data analysis and handling datasets.
- Matplotlib : Used for creating graphs and charts.

TAISON EDUCATION

Algorithm :

1. Start the program.
2. Import Numpy, Pandas, and Matplotlib libraries.
3. Create a Numpy array and perform basic operations.
4. Create a Pandas DataFrame and display it.
5. Plot the array values using Matplotlib.
6. Display the results.
7. Stop the program.

Program :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Numpy Operations
arr = np.array([10, 20, 30, 40, 50])
```

```
print ("Numpy Array:", arr)
print ("Sum:", np.sum(arr))
print ("Mean:", np.mean(arr))
print ("Maximum Value:", np.max(arr))

# Pandas Operations
data = {
    "Name": ["Amit", "Priya", "Rahul"],
    "Marks": [85, 90, 50]
}

df = pd.DataFrame(data)

print ("\nPandas Dataframe")
print (df)

# Matplotlib Plot
plt.plot(arr, marker='o')
plt.title("Numpy Array Values")
plt.xlabel("Index")
plt.ylabel("Values")
plt.grid(True)
plt.show()
```

Result :

Thus the Python program to import commonly used Python libraries and demonstrate their basic operations was implemented and executed successfully.

Pearson EDUCATION

Practical - 3

Aim :- Write a Python program to perform data collection and data handling on a dataset.

Theory :-

Data preprocessing is an important step in Machine Learning. Before building a model, the collected data must be cleaned and organized. Data collection involves gathering data from different sources. Data cleaning removes or replaces missing values, duplicate records, and incorrect data. Data handling refers to organizing, modifying, and preparing the dataset for analysis.

Algorithm :-

1. Start the program.
2. Import the Pandas library.
3. Create a sample dataset.
4. Display the original dataset.
5. Check for missing values.
6. Replace missing values using the mean.
7. Display the cleaned dataset.
8. Stop the program.

Program :-

```
import pandas as pd
import numpy as np

# Create Sample Dataset
data = {
    "Name" : ["Amit", "Priya", "Rahul", "Sneha"],
    "Age" : [20, np.nan, 22, 21],
    "Marks" : [85, 90, np.nan, 88]
}
```

```
df = pd.DataFrame(data)

# Display original Dataset
print("Original Dataset")
print(df)

# check Missing values
print("\nMissing values")
print(df.isnull())

# Data cleaning
df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())

# Display cleaned Dataset
print("\ncleaned Dataset")
print(df)
```

Result :

Thus, the Python program to perform data collection, data cleaning, and data handling on a dataset was implemented and executed successfully.

Practical - 4

Aim :- Write a Python program to implement data normalization and standardization.

Theory :-

Data preprocessing is an important step in Machine Learning. It helps improve the performance of machine learning models.

- Normalization scales the data to a fixed range, usually 0 to 1.
- Standardization transforms the data so that it has a mean of 0 and a standard deviation of 1.

Normalization is useful when features have different ranges, while standardization is commonly used in algorithms such as Logistic Regression, SVM, and K-Means.

MPSTME EDUCATION

Algorithm :-

1. Start the program
2. Import the required libraries.
3. Create a sample dataset.
4. Apply Min-Max Normalization
5. Apply Standardization
6. Display the original, normalized, and standardized data.
7. Stop the program.

Program :-

```
from sklearn.preprocessing import MinMaxScalar
from sklearn.preprocessing import StandardScalar
import pandas as pd
```

```
# Sample Dataset
data = pd.DataFrame({
    "Salary": [40000, 48000, 60000, 70000, 75000]
})

print("Original Dataset")
print(data)

# Normalization
normalizer = MinMaxScaler()
data["Normalized"] = normalizer.fit_transform(data[["Salary"]])

# Standardization
standardizer = StandardScaler()
data["Standardized"] = standardizer.fit_transform(data[["Salary"]])

print("\nDataset After Normalization and Standardization")
print(data)
```

Result :-

Thus, the Python program to implement data normalization and standardization was implemented and executed successfully.

Practical No. 5

Aim :
Write a Python Program to perform feature selection and feature engineering on a dataset.

Theory :
Feature selection means selecting the most important features from a dataset and removing unnecessary features.
Feature Engineering means creating a new useful feature from existing features.
In this program :

- We create a student dataset.
- We create a new feature called Total_Marks from three subject marks.
- We use SelectBest to select the most useful features.

Program :

```
import pandas as pd
from sklearn.feature_selection import SelectKBest, f_
classif

# Create dataset
data = {
    "Study_Hours" : [2,3,4,5,6,7,8,9],
    "Attendance" : [50,55,60,65,70,75,80,85],
    "Maths" : [45,50,55,60,65,70,75,80],
    "Science" : [60,65,70,75,80,85,90,95],
    "English" : [40,45,50,55,60,65,70,75],
    "Result" : [1,0,0,1,1,1,1,1]
}
```

```
df = pd.DataFrame(data)

print("Original Dataset:")
print(df)

# Create a new feature
df["Total_Marks"] = df["Maths"] + df["Science"] + df
["English"]

print("\nDataset After Feature Engineering:")
print(df)

x = df[
    ["Study_Hours", "Attendance", "Maths", "Science",
     "English", "Total_Marks"]
]

y = df["Result"]

# Select top 3 features
selector = SelectKBest(score_func=f_classif, k=3)

x_selected = selector.fit_transform(x, y)

# Display selected
selected_features = x.columns[selector.get_support()]

print("\nSelected features:")
print(selected_features.tolist())

print("\nSelected Featured Data:")
print(x_selected)
```

Result :-

Thus, the Python program to perform feature selection and feature engineering on a dataset was implemented and executed successfully.

Practical No. 6

Aim :
Write a Python program to perform Exploratory Data Analysis (EDA) and data visualization using Matplotlib and Seaborn.

Theory :
Exploratory Data Analysis (EDA) is the process of examining and understanding a dataset before applying Machine Learning algorithms. It helps identify:

- Dataset size
- Data types
- Missing values
- Statistical summary
- Distribution of data
- Relationship between features
- Outliers

Matplotlib is used for basic plotting, while Seaborn provides advanced statistical visualizations. EDA improves data quality and helps in selecting the appropriate Machine Learning model.

Algorithm :

1. Import the required libraries.
2. Load the Iris dataset.
3. Display the first five records.
4. Display the shape and information of the dataset.
5. Display statistical summary.
6. Check missing values.
7. Create a scatter values.
8. Create a Histogram.
9. Create a Box plot.
10. Display all graphs.

Program :

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as ~~plt~~ plt

# Load Iris Dataset
df = sns.load_dataset("iris")

# Display first five rows
print("First five Records")
print(df.head())

# Display dataset shape
print("\nDataset shape")

# Display dataset information
print("\nDataset Information")
print(df.info())

# Statistical summary
print("\nstatistical summary")
print(df.describe())

# Check Missing values
print("\nMissing values")
print(~~df.describe "\nMissing values")~~ df.isnull().sum())

# Scatter Plot
plt.figure(figsize=(6,4))
sns.scatterplot(x="sepal_length",
                y="petal_length",
                hue="species",
                data=df)
```

```
plt.title("Scatter Plot")
plt.show()

# Histogram
plt.figure(figsize=(6,4))
sns.histplot(df["sepal length"], bins=10, color="skyblue")
plt.title("Histogram of Sepal Length")
plt.xlabel("Sepal Length")
plt.ylabel("Frequency")
plt.show()

# Box Plot
plt.figure(figsize=(6,4))
sns.boxplot(x="species",
            y="petal_length",
            data=df)
plt.title("Box Plot")
plt.show()
```

Result :

Thus, the Python program to perform Exploratory Data Analysis (EDA) and data visualization using Matplotlib and Seaborn was implemented and executed successfully.

Practical No. 7

Aim :
Write a Python program to implement Linear Regression and predict continuous values.

Theory :
Linear Regression is a supervised machine learning algorithm used to predict continuous numerical values. It finds the best fit straight line that represents the relationship between an independent variable (x) and a dependent variable (Y).
For example :
- Predicting salary based on years of experience.
- Predicting house price based on area.
- Predicting marks based on study hours.

In this practical, we predict Salary using Years of Experience.

Algorithm :
1. Import the required libraries.
2. Create a sample dataset.
3. Separate input (x), and output (y).
4. Create the Linear Regression model.
5. Train the model.
6. Predict the salary for a new experience value.
7. Display the prediction.
8. Plot the regression line.

Program :

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression.
```

```
# Sample Dataset
data = {
    "Experience" : [1, 2, 3, 4, 5, 6],
    "Salary" : [40000, 50000, 60000, 70000, 80000, 90000]
}

df = pd.DataFrame (data)

# Input and Output
X = df[["Experience"]]
y = df[["Salary"]]

# Create Linear Regression Model
model = LinearRegression ()

# Train the Model
model.fit (X, y)

# Predict Salary for 7 Years Experience
prediction = model.predict (pd.DataFrame {{'Experience': [7]}
})

# Plot Regression Line
plt.scatter (X, y, color = "blue", label = "Actual Data")
plt.plot (X, model.predict (X), etc color = "red", label = "Regression
Line")

plt.title ("Linear Regression")
plt.xlabel ("Years of Experience")
plt.ylabel ("Salary")
plt.legend ()

plt.show ()
```

Result :

Thus, the Python program to implent Linear Regression and predict continuous values was implemented and executed successfully.

Practical No. 8

Aim :-
Write a Python program to implement Logistic Regression for classification based on input features.

Theory :-
Logistic Regression is a supervised Machine Learning algorithm used for classification. It predicts the class / category of an observation based on one or more input features.

In this practical, we classify students into :
- 0 → Fail
- 1 → Pass

based on their Study Hours and Attendance.

Algorithm :-
1. Import the required libraries.
2. Create a student dataset.
3. Separate input features and target variable.
4. Split the dataset into training and testing data.
5. Create a Logistic Regression model.
6. Train the model.
7. Predict the classes for test data.
8. Calculate model accuracy.
9. Predict the class of a new student.
10. Display the result.

Program :-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score

# Create Dataset
data = {
    "study_Hours": [1, 2, 2, 3, 4, 5, 6, 7, 8],
    "Attendance": [40, 45, 50, 55, 60, 65, 70, 75, 80, 85],
    "Result": [1, 0, 0, 0, 0, 1, 0, 1, 1, 1]
}

df = pd.DataFrame(data)

print("original Dataset:")
print(df)

# Input features
x = df["study_Hours", "Attendance"]

# Target Variable
y = df["Result"]

# Split Dataset
x_train, x_test, y_train, y_test = train_test_split(x, y,
    test_size = 0.2, random_state = 42
)

# Create Logistic Regression Model
model = LogisticRegression()

# Train the model
model.fit(x_train, y_train)

# Predict Test Data
y_pred = model.predict(x_test)
```

```
# Calculate Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nActual values:")
print(y_test.values)

print("\nPredicted values:")
print(y_pred)

print("\nAccuracy:", accuracy * 100, "%")

# Predict for a new student
new_student_data = [[5, 80]]
# Convert new_student to a DataFrame with appropriate column names
new_student_df = pd.DataFrame(new_student_data, columns=X.columns)

prediction = model.predict(new_student_df)

if prediction[0] == 1:
    print("\nNew student Result: PASS")
else:
    print("\nNew student Result: FAIL")
```

Result :-

Thus, the Python program to implement Logistic Regression for classification based on input features was implemented and executed successfully.

Practical No. 9

Aim :
Write a Python program to implement the Decision Tree classification ~~algo~~ algorithm on a dataset.

For this practical, we will use the Breast Cancer dataset available in scikit-learn. The Decision Tree will classify the tumor into two categories :

- Malignant
- Benign

This is a little more advanced than the Iris example because the dataset contains 30 input features.

1. Theory

A Decision Tree is a supervised Machine Learning algorithm used for classification and regression.

For classification, the Decision Tree divides the dataset into smaller groups using conditions on input features. It keeps splitting the data until it reaches a final decision called a leaf node.

For example :

Feature <= Threshold ?

Yes No

Another Test Class

Class A Class B

The tree automatically determines the best features and splitting conditions from the training data.

2. Dataset Used

We use Scikit-learn's built-in Breast Cancer Wisconsin dataset.

It contains:

- 569 samples
- 30 numerical input features
- 2 target classes.

Some features are:

- Mean Radius
- Mean Texture
- Mean Perimeter
- Mean Area
- Mean Smoothness
- Mean Compactness
- Mean Concavity
- Mean Symmetry
- Worst Radius
- Worst Area
- etc.

Target Classes

0 → Malignant

1 → Benign

Algorithm :-

1. Import required libraries.
2. Load the Breast Cancer dataset.
3. Convert the dataset into a Dataframe.
4. Display the first five records.
5. Check the dataset shape.

6. Separate input features and target variable.
7. Split the dataset into training and testing sets.
8. Create a Decision Tree Classifier.
9. Train the model.
10. Predict the test data.
11. Calculate accuracy.
12. Display the confusion matrix.
13. Display the classification report.
14. Predict a new sample.
15. Visualize the Decision Tree.
16. Stop

Program :-

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_
tree
frome sklearn_metrics import (
        accuracy_score,
        confusion_matrix,
        classification_report,
        ConfusionMatrixDisplay
)

cancer = load_breast_cancer()

df = pd.DataFrame(
    cancer.data,
    columns = cancer.feature_names
```

```
)
df["target"] = cancer.target

print("First Five Records:")
print(df.head())

print("\nDataset Shape:")
print(df.shape)

print("\nDataset Shape:")
print(df.shape)

print("\nTarget Classes:")
print(cancer.target_names)

X = cancer.data
y = cancer.target

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size = 0.20,
    random_state = 42,
    stratify = y
)

print("\nTraining Samples:", len(X_train))
print("Testing Samples:", len(X_test))

model = DecisionTreeClassifier(
    criterion = "entropy",
    max_depth = 4,
    random_state = 42
```

Pearson EDUCATION

```
)
model.fit(X_train, y_train)
print("\nDecision Tree Model Trained Successfully.")

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:")
print(accuracy * 100, "%")

cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Display Confusion Matrix
disp = ConfusionMatrixDisplay(
        confusion_matrix = cm,
        display_label = cancer, target_names
)

disp.plot()
pH.title("Confusion Matrix")
pH.show()

print("\nClassification Report:")
print(
    classification_report(
    y_test,
    y_pred,
    target_names cancer.target_names
    )
```

```
)

new_sample = [x_test[0]]

prediction = model.predict(new_sample)

print("\nNew Sample Prediction:")

if prediction[0] == 0:
    print("Malignant")
else:
    print("Benign")

plt.figure(figsize=(20, 10))

plot_tree(
    model,
    feature_names = cancer.feature_names,
    class_names = cancer.target_names,
    filled = True,
    rounded = True,
    fontsize = 8
)

plt.title("Decision Tree Classification")
plt.show
```

Result :

Thus, the Python program to implement the Decision Tree classification algorithm on a dataset was successfully implemented and executed.